CSC458H1: Computer Networks — Fall 2026

Programming Assignment 2: Intra-Domain Routing in Mininet: Distance Vector & Link State

Due Date: Friday, Nov 13, 11:59 PM

Please check out the assignment policy page on Quercus for details on CSC458 policy on collaboration and code use. Note that the assignment is expected to be submitted in 2 different milestones:

- Part 0 & I is due **Friday, Nov 6, 11:59 PM**
- The final due date is **Friday, Nov 13, 11:59 PM**

Overview

PA1 built the forwarding/data-plane foundation: Ethernet/ARP, IPv4 forwarding, static routes, CIDR, and longest-prefix matching (LPM). In PA2 you will *reuse* that forwarding knowledge rather than implement it again. Your completed PA1 `prefix_tools.py` is carried into this repository, while the new programming work focuses on the **control plane**: how routers dynamically discover the next hop and metric to associate with each remote destination prefix.

You will implement both major families of intra-domain routing:

1. **Distance Vector (DV)**: routers exchange estimates of path cost and apply the Bellman–Ford recurrence;
2. **Link State (LS)**: routers flood link-state advertisements, construct a link-state database (LSDB), and run Dijkstra's shortest-path algorithm.

Before those algorithms, Part 0 is an extended Mininet lab. It adapts the complete Lancaster University Mininet introduction to the CSC458 VM: you will understand Mininet's hosts/switches/links/controllers, use the `mn` launcher and Mininet CLI, construct and extend topologies in Python, measure connectivity and bandwidth, add traffic-control constraints, work with multiple topologies, and briefly exercise a remote OpenFlow controller. You will then switch from Ethernet-switch emulation to the Linux-router namespaces used by the rest of PA2.

The routing protocol itself runs as a real Python daemon in each Mininet router namespace. Control messages are carried in UDP datagrams over router-to-router links. A supplied adapter translates your computed routes into Linux `ip route` entries. Ordinary `ping`, `traceroute`, and application traffic therefore follow the routes produced by your DV/LS code.

Learning objectives

By the end of the assignment, you should be able to:

- reuse PA1's CIDR/LPM implementation as part of a larger forwarding stack rather than duplicate it;
- explain how Mininet uses Linux namespaces, software switches/bridges, and virtual links to emulate a network;
- create custom Mininet topologies in Python and use `mn`, `net`, `links`, `pingall`, and per-node shell commands;
- measure connectivity/latency with `ping` and bandwidth with `iperf3`;

- apply bandwidth, delay, and loss constraints with `TCLink`;
- distinguish a Mininet/OpenFlow controller from the distributed routing daemons used in PA2;
- distinguish routing (control plane) from forwarding (data plane) in a running Linux network;
- implement distributed Bellman–Ford routing using only neighbor information;
- explain and reproduce count-to-infinity behavior, split horizon, and poison reverse;
- originate, flood, and suppress stale link-state advertisements;
- construct an LSDB and derive next hops using Dijkstra's algorithm;
- react to link failures, recovery, and administrative cost changes; and
- connect control-plane convergence to actual Linux FIB state and packet delivery.

1 Architecture: what is real, and what is provided?

Each PA2 router R_i has an attached host h_i on the stub network `10.`i`.0.0/24`. For example, R_1 owns prefix `10.1.0.0/24`; its router interface is `10.1.0.1`, and host `h1` is `10.1.0.2`. Router-to-router links use private /30 networks assigned deterministically by the supplied code.

A typical routing topology is conceptually

```
h1 -- R1 ---- R2 ---- R3 ---- R4 -- h4
      |        |       |        |
   Linux    Linux   Linux    Linux
   router   router  router   router
```

Part 0 first uses ordinary Mininet hosts and OvS switches because that is the clearest way to learn the Mininet abstraction. Parts I–III use a different Mininet pattern: Linux network namespaces act as routers with IPv4 forwarding enabled. There is no OpenFlow controller in the DV/LS experiment topology.

Your routing protocol has two layers:

1. **Student protocol core**: `dv_core.py` or `ls_core.py`. It sees only local link notifications and control messages received from one-hop neighbors.
2. **Runtime**: `daemon.py`, `linux_routes.py`, and `mininet_lab.py`. The runtime sends/receives UDP control messages, detects interface carrier changes, and installs the routes computed by your core into the Linux FIB.

   Design rule. PA1 owns CIDR/LPM/static-forwarding logic. Your PA2 must not introduce a second implementation of LPM. The Linux-FIB adapter imports `csc458_pa2.prefix_tools.canonical_prefix`; your new algorithmic work is DV and LS route computation.

   Design rule. Your protocol core must not import Mininet, inspect the global topology JSON, call the shortest-path oracle, invoke shell commands, or otherwise learn information unavailable to a real routing process. Hidden tests check protocol logic independently of Mininet.

The routing metric of a link is a logical IGP cost. It is not the same as propagation delay. The supplied PA2 Mininet topology uses a small emulation delay so that control traffic is real, while route selection uses the integer costs in the topology file.

2 Setup and PA1 carry-forward

Use the CSC458 Ubuntu VM. First copy your completed PA1 prefix module into this repository and check it:

```
cp /path/to/your/PA1/src/csc458_pa1/prefix_tools.py src/csc458_pa2/prefix_tools.py
python3 tools/pa1_prefix_smoke.py
```

You are not being asked to write these functions again.

For unit tests:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export PYTHONPATH="$PWD/src"
pytest -q -m "not mininet"
```

or run `./grade_local.sh`. The deterministic harness is for fast protocol testing; experiment results must come from Mininet.

Verify the VM with

```
sudo python3 tools/check_environment.py
sudo mn --test pingall
```

If Mininet crashes and leaves stale state, use `sudo mn -c`.

Repository layout

```
CSC458_PA2_Mininet_student_release/
  mininet_tutorial/
    README.md # Part 0: complete CSC458/Lancaster tutorial
    topology.py # EDIT during Part 0
    MN.md # supplied mn command reference
    Lancaster_original_README.md
    Lancaster_topology_original.py
  src/csc458_pa2/
    prefix_tools.py # COPY your completed PA1 file here
    dv_core.py # EDIT: Part I
    ls_core.py # EDIT: Part II
    core_api.py # provided API
    daemon.py # provided UDP runtime
    linux_routes.py # provided FIB adapter; reuses PA1 prefix code
    mininet_lab.py # provided PA2 Mininet runtime
    routing_cli.py # provided CLI extensions
    core_harness.py # provided unit-test harness
    topology.py # provided PA2 topology/event parser
    addressing.py # provided addressing
    oracle.py # checker; DO NOT import from your core
  topologies/
  tests/
  tools/
  report/answers_template.md
```

For submission, modify only `mininet_tutorial/topology.py`, `dv_core.py`, `ls_core.py`, your copied PA1 `prefix_tools.py`, your report, and any additional tests you add. The copied PA1 module is a prerequisite/carry-forward artifact, not a new PA2 implementation task.

3 Part 0 – Mininet Introduction and the Forwarding/Control-Plane Bridge [20 points]

3.1 Understanding the topology

The starter `topology.py` creates one Mininet host (`h1`), one switch (`s1`), and one virtual link. Mininet emulates networks primarily using Linux network namespaces, software switches/bridges, and virtual Ethernet interfaces.

Four fundamental components:

1. **Hosts:** act like computers. A host has its own network namespace, interfaces, IP/MAC addresses, processes, and can run ordinary Linux commands.
2. **Switches:** software forwarding elements. Open vSwitch (OvS) can behave as a bridge and is OpenFlow-capable.
3. **Links:** virtual connections between nodes; Mininet creates virtual interfaces as link endpoints.
4. **Controllers:** the control plane for OpenFlow-enabled switches. A controller can be created by Mininet or can run remotely at an IP address and TCP port.

Read `topology.py` and identify the calls that create each node/link and the `topos` dictionary that exposes the topology to `mn -topo`.

3.2 Run a custom topology and learn the CLI

From `mininet_tutorial/` run

```
sudo mn --custom ./topology.py --topo tutorialTopology --switch ovs
```

`-switch ovs` selects Open vSwitch. Explore `mn -h` and `mininet_tutorial/MN.md` for more information. At the `mininet>` prompt use `help`, `nodes`, `net`, `links`, `exit`, and per-node commands such as

```
h1 echo "Hello World!"
h1 ip addr
s1 ip link
```

A Mininet CLI line prefixed by a node name executes that command in that node's context. This ability is central to later PA2 experiments, where you will run `ip route`, `ping`, `traceroute`, `tcpdump`, and routing daemons in different namespaces.

After exiting, run `sudo mn -c`.

3.3 Extend the topology

Modify `topology.py` so `s1` has five hosts, `h1` through `h5`. Verify with:

```
net
links
pingall
h1 ping -c 5 h4
```

You should see five host-switch links and, without intentional loss, `pingall` should report 0% dropped (20/20 host-to-host probes).

Expand to two switches: Attach `h1`–`h5` to `s1`, `h6`–`h10` to `s2`, and connect `s1` to `s2`. Verify with `net`, `links`, and `pingall`. Port numbering can vary, but the graph must be the same. With unconstrained links, `pingall` should report full reachability (90/90 host-to-host probes).

Bonus/refactoring. Use ordinary Python lists/loops/helpers to avoid repetitive host/link declarations. The topology is a Python program, not a static configuration file.

After each run, use `sudo mn -c` if cleanup is required.

3.4 Basic testing with ping and iperf3

Mininet shortcuts such as `pingall`, `net`, and `links` are useful, but host commands expose richer measurements.

Run

```
h1 ping -c 5 h8
```

to inspect packet loss and RTT rather than only a Boolean connectivity result. Record packet loss and min/avg/max RTT. The important observation is that a first packet can be slower. Why? Connect this observation to ARP resolution (PA1) and Ethernet-switch learning.

For bandwidth, run an `iperf3` server and client:

```
h8 iperf3 -s &
h1 iperf3 -c h8 -p 5201
```

The unconstrained software-only path can report very high throughput; treat this as an emulation baseline, not physical-link capacity.

3.5 Multiple topologies in one file

Add another `Topo` subclass to the same file and register both names in `topos`, e.g.

```
topos = {
    'tutorialTopology': (lambda: TutorialTopology()),
    'newTopology': (lambda: MyNewExampleTopology()),
}
```

Launch each by changing `-topo`.

3.6 Add realism with TCLink

Real networks have finite bandwidth, delay, and loss. Mininet's `TCLink` applies Linux traffic-control constraints to virtual links.

Add:

```
from mininet.link import TCLink
```

You can experiment with TCLink by specifying a link's bandwidth, delay, and loss. For example

```
self.addLink(h1, s1, cls=TCLink, bw=100)
self.addLink(h2, s1, cls=TCLink, delay='75ms')
self.addLink(h3, s1, cls=TCLink, loss=5)
```

Set the `s1-s2` link to **50 Mbit/s**, **30 ms delay**, and **10% loss**. Re-run `ping` and `iperf3`. Do not expect one exact TCP throughput under loss.

3.7 Remote controller / SDN orientation

With `-switch ovs`, Mininet uses Open vSwitch, an OpenFlow-capable software switch. OpenFlow separates a programmable controller (control plane) from switch forwarding behavior (data plane). A remote controller is selected with `-controller remote` and may be given an IP/port.

For a local reference controller, open a **second VM terminal** and try:

```
controller ptcp:6633
```

Then launch:

```
sudo mn --custom ./topology.py --topo tutorialTopology \
       --switch ovs --controller remote,ip=127.0.0.1,port=6633
```

Try connectivity with the controller running, then with it unavailable.

> **Important.** This SDN stage teaches what Mininet can emulate; PA2's DV/LS routers do **not** use OpenFlow. Their control plane is your distributed Python routing daemon, and their forwarding plane is the Linux FIB.

3.8 Bridge to the PA2 Linux-router topology

Run

```
sudo python3 tools/mininet_intro.py
```

This creates `R1-R2-R3` with one stub host behind each router and no dynamic routing daemon. Inspect `ip addr` and `ip route`. Initially `h1` cannot reach `10.3.0.2` because only directly connected prefixes are present.

Install correct static routes manually, using the forwarding concepts from PA1. For example:

```
r1 ip route add 10.3.0.0/24 via 10.254.0.6 dev r1-r2
```

Determine the remaining routes, verify all-pairs reachability, and run `h1 traceroute -n 10.3.0.2`. Then fail `r2-r3` with `link r2 r3 down`. The manually configured FIB entries remain even when their next-hop path becomes unusable. Restore the link with `link r2 r3 up`.

This is the motivation for Parts I–III: DV/LS will compute and update remote-prefix forwarding state dynamically.

> **Deliverable.** Complete Part 0 of `report/answers_template.md` and submit your final `mininet_tutorial/topology.py`. Include evidence from topology construction, ping/iperf3, TCLink, the remote-controller orientation, and the Linux-router/static-route bridge.

4 Protocol API

Both cores implement the following conceptual callbacks:

```
on_link_change(neighbor: str, cost: int | None)
on_control(src: str, message: ControlMessage)
on_periodic()
get_route(prefix: str) -> Route | None
all_routes() -> dict[str, Route]
```

An integer cost indicates an active local adjacency. `None` indicates that the interface is no longer usable. The daemon detects physical link up/down by polling the Linux interface carrier state. Administrative cost changes are sent by the supplied experiment runner.

Your core sends a message only through

```
self.send_control(neighbor, ControlMessage(...))
```

The daemon serializes this object to JSON and sends it in a UDP datagram to that directly connected neighbor on port 4582.

5 Part I – Distance Vector Routing [30 points]

For router x, destination prefix p, and neighbor set $N(x)$, implement

$$D_x(p) = \min_{v \in N(x)} \{c(x, v) + D_v(p)\}.$$

Router x does not know the network graph. It knows only its own prefix, local link costs, and vectors received from its neighbors.

Use

$$\texttt{INFINITY} = 16.$$

Metrics greater than or equal to 16 are unreachable. Cap additions at 16.

Each router always originates its own stub prefix with metric 0. A typical control message is conceptually

```
{
  "protocol": "DV",
  "payload": {
    "type": "dv",
    "vector": {
      "10.1.0.0/24": 0,
      "10.4.0.0/24": 4
    }
  }
}
```

Implement all TODOs in `src/csc458_pa2/dv_core.py`.

5.1 Required behavior

1. **Bellman–Ford recomputation.** Build the best route to every known prefix from local link costs and received vectors. Equal-cost paths use the lexicographically smaller next-hop router ID.
2. **Triggered updates.** If a received vector or local link event changes your routing table, advertise the new vector immediately.
3. **Periodic updates.** `on_periodic()` sends a complete current vector to all active neighbors.

4. **Link failure.** Remove information learned from a failed neighbor and recompute.
5. **Basic DV.** Advertise every known metric normally.
6. **Split horizon.** Do not advertise a route back to the neighbor that is currently its next hop.
7. **Poison reverse.** Advertise that route back to its next hop with metric 16 instead of omitting it.
8. **Robust parsing.** Ignore malformed fields rather than crashing.

The mode is selected by the runner:

```
sudo python3 tools/run_mininet.py line4.json --protocol DV --dv-mode basic
sudo python3 tools/run_mininet.py line4.json --protocol DV --dv-mode split_horizon
sudo python3 tools/run_mininet.py line4.json --protocol DV --dv-mode poison_reverse
```

After convergence, inspect the actual Linux FIB:

```
sudo python3 tools/run_mininet.py line4.json --protocol DV \
    --dv-mode poison_reverse --interactive
```

then at the `csc458>` prompt:

```
routes R1
status R1
h1 ping -c 2 10.4.0.2
h1 traceroute -n 10.4.0.2
```

6 Part II – Link State Routing [30 points]

Each router originates an LSA containing its identity, sequence number, owned prefix, and currently active local adjacencies. A conceptual LSA is

```
{
  "protocol": "LS",
  "payload": {
    "type": "lsa",
    "origin": "R1",
    "seq": 7,
    "prefix": "10.1.0.0/24",
    "links": {"R2": 1, "R3": 4}
  }
}
```

Implement all TODOs in `src/csc458_pa2/ls_core.py`.

6.1 Required behavior

1. **Origination.** Increment your own sequence number whenever local adjacency state/cost changes and during periodic refresh. Store the new self-LSA locally before flooding it.
2. **Flooding.** Forward a newly accepted LSA to all active neighbors except the neighbor from which that copy arrived. Do not alter the origin, sequence number, prefix, or link contents while forwarding.
3. **Freshness.** For each origin, accept only a strictly newer sequence number. Duplicate/stale LSAs must not trigger reflooding.

4. **New-neighbor synchronization.** When a new adjacency appears, send the new neighbor your current LSDB so it need not wait for every origin's next periodic refresh.
5. **Self-LSA fight-back.** If a supposedly self-originated LSA returns with a sequence number newer than your own, advance beyond it and re-originate your correct local state.
6. **Consistent graph construction.** Treat link $u - v$ as usable only when the current LSAs from both u and v advertise one another with the same cost. This prevents a stale half-link from being used while failure information is propagating.
7. **Dijkstra.** Compute least-cost paths without using a graph library. For equal-cost paths, choose the smaller first hop; if the first hop is also equal, choose the lexicographically smaller complete router path.

The route to an advertised prefix is the route to the LSA origin that owns that prefix.

Run LS with

```
sudo python3 tools/run_mininet.py line4.json --protocol LS
```

7 Part III – Convergence and Failure Experiments [15 points]

Your experiments must use Mininet, not the unit-test harness. The supplied runner creates a `run/.../results.json` file containing convergence timing and control-message counters.

7.1 A. Forwarding correctness

For both DV (poison reverse) and LS, use `weighted_mesh.json`. Show one nontrivial source/destination pair for which:

- the protocol status reports the expected next hop and metric;
- `ip route` contains the corresponding remote /24 route;
- `ping` succeeds; and
- `traceroute` follows the expected router path.

7.2 B. Failure convergence

Run `failure_alt.json` with DV-basic, DV-poison-reverse, and LS. Record initial convergence, reconvergence after R2–R4 fails, and the control-message/byte delta reported for the event.

Explain the R1-to-R4 path before and after failure. Do not over-interpret sub-second timing differences: Mininet runs on a shared OS scheduler. Focus on protocol behavior and repeat a run if a result appears anomalous.

7.3 C. Count to infinity

Run

```
sudo python3 tools/run_mininet.py count_to_infinity.json \
    --protocol DV --dv-mode basic --capture
```

and repeat with poison reverse.

The generated per-router PCAPs are stored under the run directory. Open a capture in Wireshark or inspect it with `tcpdump`. Identify at least three successive advertised metrics that demonstrate count-to-infinity in basic DV, and explain why poison reverse changes the behavior.

7.4 D. Cost change

Run `cost_change.json` with DV and LS. Its scripted event raises the cost of link R2–R4 from 1 to 8. State R1's route to `10.4.0.0/24` before and after the cost increase. Explain how each protocol learns and disseminates the change.

7.5 E. Synthesis

In 200–300 words, compare DV and LS using your own measurements and observations. Address:

- what local/global information a router stores;
- what is carried in a control message;
- convergence after failure/cost change;
- control overhead; and
- the failure/pathology mechanisms you observed.

8 Submission

Submit the following for Milestone 1:

1. `mininet_tutorial/topology.py`;
2. your copied `src/csc458_pa2/prefix_tools.py`;
3. `src/csc458_pa2/dv_core.py`;
4. report with Part 0 completed

Submit all required files for the final submission:

1. `mininet_tutorial/topology.py`;
2. your copied `src/csc458_pa2/prefix_tools.py`;
3. `src/csc458_pa2/dv_core.py`;
4. `src/csc458_pa2/ls_core.py`;
5. Fully completed report

Do not submit the `run/` directory or generated PCAPs unless explicitly requested. The PA1 prefix file is submitted only because PA2 imports it; no new PA2 marks are awarded for reimplementing LPM.

A Appendix: Useful commands

Command	Purpose
`./grade_local.sh`	Run non-Mininet public tests.
`sudo python3 tools/mininet_intro.py`	Mininet/static-routing warm-up.
`sudo python3 tools/run_mininet.py line4.json -protocol DV`	Run DV in Mininet.
`sudo python3 tools/run_mininet.py line4.json -protocol LS`	Run LS in Mininet.
`-interactive`	Enter the CSC458/Mininet CLI after convergence.
`-capture`	Capture UDP/4582 control traffic into per-router PCAP files.
`routes R1`	Show R1's Linux forwarding table in interactive mode.
`status R1`	Show R1's routing-daemon state.
`link r1 r2 down`	Physically fail the Mininet link; daemon detects carrier loss.
`cost R1 R2 5`	Change the logical IGP cost without changing physical delay.
`converged`	Check whether current protocol routes match expected shortest paths.
`sudo mn -c`	Clean up a crashed Mininet run.